
Sparse ActionGen: Accelerating Diffusion Policy with Real-time Pruning

Kangye Ji¹ Jianbo Zhou¹ Yuan Meng² Ye Li¹ Hanyun Cui¹ Zhi Wang¹

Abstract

Diffusion Policy has dominated action generation due to its strong capabilities for modeling multi-modal action distributions, but its multi-step denoising processes make it impractical for real-time visuomotor control. Existing caching-based acceleration methods typically rely on *static* schedules that fail to adapt to the *dynamics* of robot-environment interactions, thereby leading to suboptimal performance. In this paper, we propose **S**parse **A**ction**G**en (**SAG**) for extremely sparse action generation. To accommodate the iterative interactions, SAG customizes a rollout-adaptive prune-then-reuse mechanism that first identifies prunable computations globally and then reuses cached activations to substitute them during action diffusion. To capture the rollout dynamics, SAG parameterizes an observation-conditioned diffusion pruner for environment-aware adaptation and instantiates it with a highly parameter- and inference-efficient design for real-time prediction. Furthermore, SAG introduces a one-for-all reusing strategy that reuses activations across both timesteps and blocks in a zig-zag manner, minimizing the global redundancy. Extensive experiments on multiple robotic benchmarks demonstrate that SAG achieves up to $4\times$ generation speedup without sacrificing performance. Project Page: <https://sparse-actiongen.github.io>.

1. Introduction

Diffusion Policy has gained substantial attention in robotic control, due to its ability to model multi-modal action distributions (Chi et al., 2023; Ze et al., 2024; Liu et al., 2024;

2025c). This capability has led to their widespread adoption in Vision-Language-Action (VLA) models (Black et al., 2024; Shukor et al., 2025; Wen et al., 2025; Liu et al., 2025b; Hou et al., 2025; Li et al., 2025), where they serve as the action expert for highly dexterous manipulation tasks. However, its massive computational burden in the denoising process makes the action frequency unable to satisfy real-time and smooth robotic control. For example, on a high-end GPU like the RTX 4090, conducting 50 diffusion denoising steps at 1 ms per step for a pick-and-place task takes 50 ms. This restricts the execution frequency to 20 Hz, well below the 50-1000 Hz needed for the Franka robotic arm.

In response to this issue, recent efforts have been made to address the inference bottleneck of Diffusion Policy. For instance, EfficientVLA (Yang et al., 2025) extends the uniform caching schedule to the action diffusion, while BAC (Ji et al., 2025) introduces a task-specific block-wise schedule to improve the caching efficiency. However, these methods typically rely on *static* caching schedules that are predefined offline and remain unchanged throughout the entire rollout process. To examine their limitations, we evaluate multiple fixed caching schedules in a leave-one-out manner: each schedule is applied to one rollout iteration while the remaining rollout uses full computation. As shown in Figure 1, *a fixed caching schedule cannot consistently accommodate the rollout dynamics*, thereby constraining the achievable trade-off between performance and efficiency.

To address this, we propose Sparse ActionGen (SAG), a rollout-adaptive acceleration method for extremely sparse action generation. To accommodate the rollout dynamics, SAG discards the fixed-schedule caching procedure and designs an adaptive prune-then-reuse mechanism that operates in concert with the robot-environment interaction. In each iteration, SAG globally identifies prunable computations and reuses cached counterparts to substitute them on the fly throughout generation.

The key to SAG lies in dynamically generating and adjusting the pruning schedule for the changing sparsity patterns of policy inference across rollout iterations, which necessitates a real-time pruner aligned with the interactive visuomotor dynamics. However, existing rule-based approaches (Ji et al., 2025; Liu et al., 2025a) typically require profiling post-forward activations, fundamentally precluding

¹Tsinghua Shenzhen International Graduate School, Tsinghua University ²Department of Computer Science and Technology, Tsinghua University. Correspondence to: Yuan Meng <yuanmeng@mail.tsinghua.edu.cn>, Zhi Wang <wangzhi@sz.tsinghua.edu.cn>.

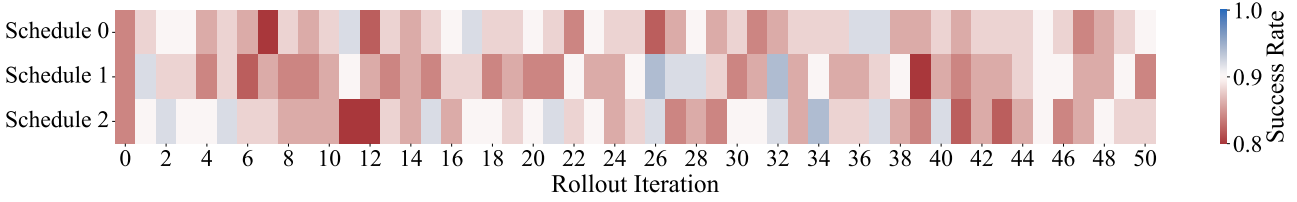


Figure 1. The performance of fixed schedules on different rollout iterations. We conduct a leave-one-out analysis on the Square task, where the robot should push or slide an object so that its center follows a square-shaped path on the table. At each x-axis position, the tested schedule is applied only to that rollout iteration and full computation is used elsewhere. Observations include: (1) A fixed schedule performs inconsistently across different rollout iterations. (2) The optimal schedules differ among different rollout iterations. Both indicate the limitations of a static caching schedule.

environment-aware adaptation. To resolve this contradiction, SAG parameterizes the pruner to be conditioned on the current observations. In this way, the pruner learns to predict the sparsity pattern a priori before the inference. However, different from previous learning-based works that predefine static computation graphs (Ma et al., 2024a; Zhu et al., 2024), performing pruning in real time inevitably introduces additional cost. To reduce the additional overhead, SAG instantiates the pruner with a parameter- and inference-efficient architectural design that serves all the blocks with a single network and requires only one forward pass for the entire denoising process.

Furthermore, we challenge the widely adopted block-wise caching paradigm (Ji et al., 2025; Ma et al., 2024a; Wimbauer et al., 2024; Yang et al., 2025; Liu et al., 2025a; Ji et al., 2026) for its largely overlooked inter-block redundancies. To address this, SAG refines the pipeline from a global allocation perspective. Specifically, for the pruning stage, SAG introduces an end-to-end global sparsity loss that guides the pruner to non-uniformly allocate computational resources across both timesteps and blocks under a strict budget. For the reusing stage, SAG proposes a one-for-all reusing strategy that reuses activations across blocks and timesteps in a zig-zag manner, motivated by the strong similarity observed in the activations across blocks (Figure 4a).

To assess the effectiveness of the proposed method, we evaluate SAG on extensive simulation benchmarks. SAG prunes over 90% computations of Diffusion Policy during the action generation throughout the entire rollout, resulting in $3.6 - 4\times$ speedup on most of the visuomotor tasks while maintaining comparable performance. In summary, our main contributions are as follows:

1. We introduce Sparse ActionGen (SAG), a rollout-adaptive acceleration method for extremely fast action generation. SAG addresses the limitations of fixed caching schedules by introducing a real-time prune-then-cache mechanism.
2. SAG proposes a rollout-adaptive diffusion pruner by conditioning it with current observations for the

environment-aware adaptation and instantiating it with a highly parameter- and inference-efficient design for real-time prediction.

3. SAG refines the prune-then-reuse pipeline from a global allocation perspective by introducing an end-to-end global sparsity loss to guide computational resource allocation and a one-for-all reusing strategy that promotes cross-block activation reuse.
4. SAG prunes over 90% of computations and delivers up to $4\times$ acceleration for action generation while maintaining comparable task performance, without updating the model parameters.

2. Related Work

Diffusion Models Caching. Caching has emerged as an effective way to accelerate diffusion inference by reusing redundant activations. Early methods such as DeepCache (Ma et al., 2024b) focus on U-Net backbones, while more recent approaches (Selvaraju et al., 2024; Ma et al., 2024a; Chen et al., 2024; Zou et al., 2025) extend caching to transformer-based diffusion models. These methods typically operate at a coarse granularity, with all the blocks sharing a uniform caching schedule (Selvaraju et al., 2024; Ma et al., 2024b), i.e., updating the cache at uniform intervals. Despite some works extending this schedule in a finer architectural granularity, they either require extra training (Ma et al., 2024a) or are specifically designed for the patterns of the image generation process (Chen et al., 2024).

Diffusion Policy Acceleration. DiT-based diffusion policies use a large multimodal transformer as the denoiser and directly denoise continuous action chunks conditioned on visual observations. One-Step Diffusion Policy (Wang et al., 2024) converts a pretrained policy into a single-step action generator. Consistency Policy (Prasad et al., 2024) and SDM Policy (Jia et al., 2024) distill a Diffusion Policy into a faster student policy by enforcing self-consistency along diffusion trajectories. VDD (Zhou et al., 2024) compresses pretrained diffusion policies into Mixture-of-Experts models using

a variational objective. In terms of caching-based methods, Streaming Policy (Høeg et al., 2024), RNR-DP (Chen et al., 2025b), and Falcon (Chen et al., 2025a) reuse partially denoised trajectories in the previous iterations, while EfficientVLA (Yang et al., 2025) adopts a uniform caching schedule, i.e., updating the cached features at fixed intervals. BAC (Ji et al., 2025) accelerates policy inference by caching intermediate action features at a fine-grained block level.

3. Methodology

In this section, we introduce SAG, an acceleration method designed to enable extremely fast action generation through rollout-adaptive sparse inference. SAG consists of two key modules: a real-time diffusion pruner that dynamically adjusts pruning schedules based on rollout dynamics, and a one-for-all reusing strategy that reuses computations across denoising steps and blocks. We first analyze the three-level redundancy on the policy model inference in Sec. 3.1. Next, we introduce the real-time diffusion pruner in Sec. 3.2 and describe the one-for-all reusing strategy in Sec. 3.3. Finally, we integrate them into a unified two-stage pipeline.

3.1. Preliminaries

To analyze the redundancy across the action diffusion process, we decompose its dynamics into three nested levels: (1) *rollout-level*, characterizing the iterative interaction between the robot and the environment; (2) *denoising-level*, where actions are generated through multi-step reverse diffusion; and (3) *block-level*, where each denoising step is realized by propagating action tokens through stacked DiT blocks. We formulate the entire process across these levels.

Rollout-Level: Robot-Environment Interaction. At rollout iteration t , the policy receives the current observation $\mathbf{o}_t$ and produces an action chunk $\mathbf{a}_t$, which is executed by the robot. The environment then evolves under its dynamics and yields the next observation:

$$\mathbf{o}_{t+1} \sim P(\mathbf{o}_{t+1} \mid \mathbf{o}_t, \mathbf{a}_t), \quad (1)$$

where $P(\cdot \mid \mathbf{o}_t, \mathbf{a}_t)$ denotes the environment’s state transition probability. This process repeats for multiple iterations until the task is completed.

Denoising-Level: Diffusion Policy Inference. In each iteration, the action $\mathbf{a}_t$ is obtained via conditional denoising diffusion (Chi et al., 2023). Starting from Gaussian noise $\mathbf{a}_t^K \sim \mathcal{N}(0, I)$, the policy performs K reverse denoising steps:

$$\mathbf{a}_t^{k-1} = f_\theta(\mathbf{a}_t^k, \mathbf{o}_t, k), \quad k = K, \dots, 1, \quad (2)$$

where f_θ is the conditional denoiser. We can therefore represent the entire multi-step denoising process, which

maps initial noise to a final action, as the policy π :

$$\mathbf{a}_t = \pi(\mathbf{o}_t) = \mathbf{a}_t^0 = f_\theta \circ f_\theta \circ \dots \circ f_\theta(\mathbf{a}_t^K, \mathbf{o}_t). \quad (3)$$

The final $\mathbf{a}_t$ serves as the control action in Eq. 1.

Block-Level: Diffusion Transformer Forward. The denoiser f_θ is parameterized by a DiT. Observations are embedded by a transformer encoder and fused into a transformer decoder with L stacked layers. Each layer l integrates temporal and observation conditioning through three blocks, i.e., self-attention (SA), cross-attention (CA), and a feed-forward network (FFN). Given hidden state $\mathbf{h}_k^{l-1}$ at denoising step k , the update is:

$$\mathbf{h}_k^l = \mathbf{h}_k^{l-1} + \text{SA}_k^l + \text{CA}_k^l + \text{FFN}_k^l. \quad (4)$$

The output of layer l is computed by summing the residual outputs of these blocks, and the final representation $\mathbf{h}_k^L$ produces $\mathbf{a}_t^{k-1}$ in Eq. 2.

3.2. Real-time Diffusion Pruner

Closed-loop action generation inherently involves multi-round interactions where a diffusion policy progressively generates an entire action trajectory. However, existing caching-based acceleration methods overlook the aforementioned rollout-level dynamics of this process, applying a single, fixed caching schedule. This raises two critical questions: (1) Is the optimal schedule consistent across all rollout iterations? (2) Does a schedule optimized for one specific rollout iteration perform well across the entire trajectory?

To investigate this, we conducted a leave-one-out evaluation on different rollout iterations. Specifically, we randomly generate three fixed caching schedules and apply them to only one rollout iteration at a time to isolate their impact on the final task success rate. As shown in Figure 1, the optimal schedule changes significantly with the rollout progress. Moreover, a schedule that excels at one iteration inevitably degrades performance at others. These findings underscore the limitations of the static schedules and suggest a rollout-adaptive caching mechanism.

The central challenge of such a mechanism is to identify prunable computations based on the sparsity pattern in each rollout iteration. To this end, SAG instantiates the pruner with two essential properties: (1) environment-aware adaptation to predict visuomotor sparsity pattern, and (2) real-time prediction to enable closed-loop visuomotor control. We elaborate on these designs below.

Environment-aware Adaptation. Motivated by the preceding analysis, we seek a pruner that dynamically generates optimal pruning schedules that adapt to the rollout dynamics. Ideally, such a schedule should mirror the sparsity pattern of

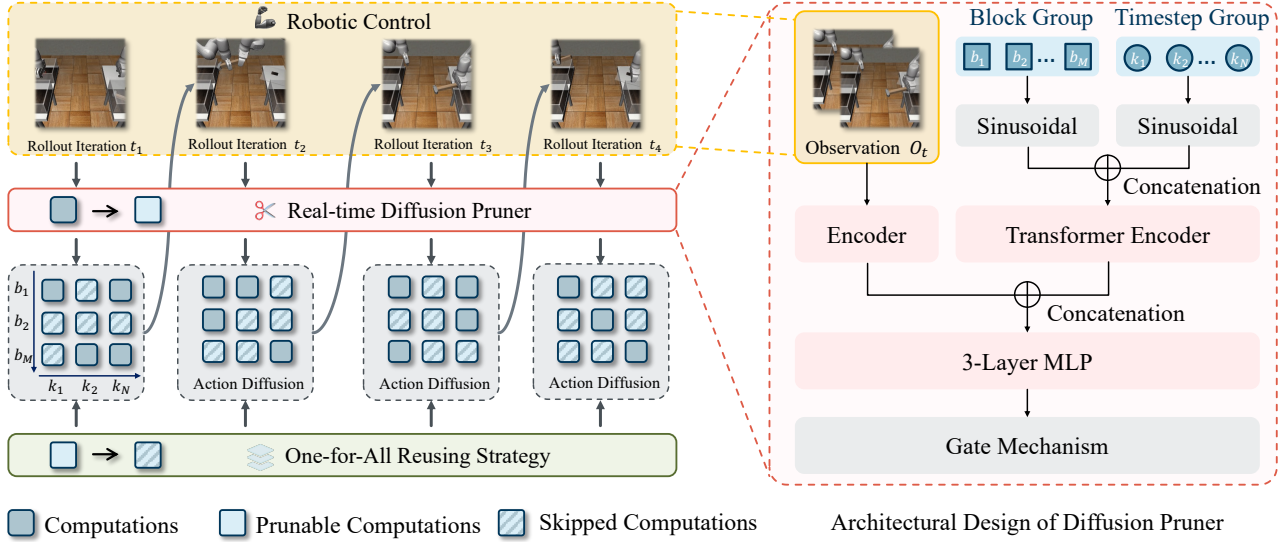


Figure 2. **Framework of Sparse ActionGen.** SAG adopts a prune-then-reuse pipeline coupled with rollout iterations. In each rollout iteration, SAG identifies the prunable computations based on the sparsity pattern predicted by the real-time diffusion pruner. During generation, SAG skips these prunable computations and substitutes them with cached activations in a one-for-all reusing strategy.

the denoising process in each interaction round. A naive idea would be to approximate this sparsity pattern by profiling activation similarities. However, such rule-based operations rely on full forward passes, negating any potential speedup. To overcome this limitation, we propose a parameterized diffusion pruner G_ψ . Instead of profiling post-forward activations for the sparsity pattern, our pruner learns to predict it. In visuomotor tasks, the computational pattern of the action generation process is highly correlated with the visual input snapped from the environment. Therefore, we condition the pruner on the current observation $\mathbf{o}_t$, enabling it to learn a direct mapping from environment inputs to the predicted sparsity pattern. To incorporate sparsity, the pruner predicts a binary pruning mask $\mathcal{M} \in \{0, 1\}^{K \times 3L}$ for all K denoising steps and $3L$ DiT blocks:

$$\mathcal{M} = G_\psi(\mathbf{o}_t), \quad (5)$$

where $\mathcal{M}_{k,b} = 1$ indicates the computation of block b at timestep k is sparse and should be pruned.

Real-time Prediction. As the introduced overhead of a parameterized pruner scales with the parameter count and number of its forward passes, SAG minimizes such cost by designing a parameter- and inference-efficient architecture for the pruner G_ψ , which generates a global pruning graph in a single forward pass, predicting the sparsity pattern for all K denoising steps and $3L$ blocks. Specifically, the pruner jointly encodes the group of timestep indices and block indices using sinusoidal positional embeddings (See Figure 3 for quantitative results). Instead of outer summation, these embeddings are concatenated to preserve their informational

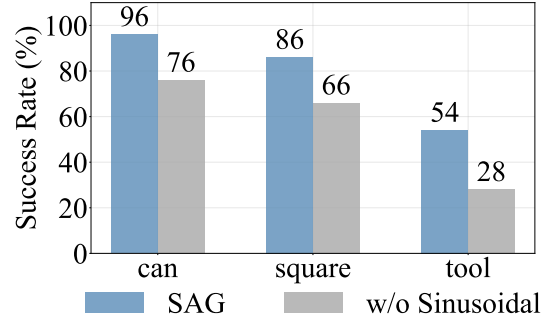


Figure 3. Success rates w/ and w/o sinusoidal positional encoding.

orthogonality. The resulting spatiotemporal coordinate representation is then processed by a Transformer encoder to capture hierarchical dependencies between temporal and structural signals.

In parallel, we adopt a visual encoder Enc_{obs} to process the current observation $\mathbf{o}_t$. The resulting observation embedding is concatenated with the spatiotemporal representation $\mathbf{z}$ to achieve a unified feature vector $\mathbf{h}$:

$$\mathbf{h} = \text{Concat}(\text{Enc}_{obs}(\mathbf{o}_t), \mathbf{z}). \quad (6)$$

This feature vector, which contains both environmental and structural information, is then projected by a three-layer MLP to produce two-dimensional logits, $\mathbf{l} \in \mathbb{R}^{K \times 3L \times 2}$. The final binary pruning mask $\mathcal{M}$ is obtained by applying an argmax operator along the last dimension of the logits:

$$\mathcal{M} = \text{argmax}(\text{MLP}(\mathbf{h})). \quad (7)$$

For end-to-end training, we employ the Straight-Through Estimator (STE) (Jang et al., 2016) to enable gradient flow through the non-differentiable argmax operation. Owing to these designs, the proposed pruner operates with exceptional efficiency, introducing only negligible computational overhead. We present the architecture illustration in Figure 2.

Optimization Objective. A key challenge is to guide the learning of the pruner G_ψ such that the pruned policy π' preserves task performance while adhering to a computational budget. Our goal is to minimize the deviation between the behaviors of the pruned policy and the ground truth action, regularized by a global sparsity constraint. The overall pruning objective is defined as:

$$\min_{\psi} \mathbb{E}_{\mathbf{o}_t \sim \mathcal{D}_{\text{ref}}} [\mathcal{L}_{\text{fidelity}}(\pi'(\cdot | \mathbf{o}_t, \mathcal{M}), \mathbf{a}_t)] + \beta \mathcal{L}_{\text{sparsity}}(\mathcal{M}). \quad (8)$$

To ensure the pruned policy π' aligns with the desired ground-truth behavior, we introduce a policy fidelity loss. Due to the global pruning objective, we directly supervise the pruned policy using a reference dataset $\mathcal{D}_{\text{ref}}$, which is constructed by uniformly sampling approximately 5% of the original training data. This encourages the pruned policy to mimic the expert actions within this representative subset. The loss is formulated as the distance between the policy’s output distribution and the ground-truth action a^* :

$$\mathcal{L}_{\text{fidelity}} = \mathbb{E}_{(o, a^*) \sim \mathcal{D}_{\text{ref}}} [\|\pi'(o, \mathcal{M}) - a^*\|], \quad (9)$$

where o is a visual state from the reference dataset. To enable non-uniform allocation of computational resources across different timesteps and blocks, we introduce an end-to-end global sparsity loss. This loss directly enforces a target global pruning rate ρ , by penalizing the deviation of the overall sparsity degree of the mask $\mathcal{M}$:

$$\mathcal{L}_{\text{sparsity}} = \left\| \frac{1}{N} \sum_{k=1}^K \sum_{b=1}^B \mathcal{M}_{k,b} - \rho \right\|. \quad (10)$$

Here, $\mathcal{M}_{k,b} \in [0, 1]$ represents the learned gating value for the b -th block at k -th denoising step, and N is the total number of prunable units, accounting for $K \times 3L$. We employ the Mean Absolute Error (MAE) loss to impose an end-to-end constraint on the model’s total computational budget, obviating the need for post-processing algorithms (Zhu et al., 2024) that select computation units based on importance scores. We visualize the real-time pruning rate throughout the rollout in Appendix A.2. By setting β to 1, the final optimization objective can be formulated as the sum of two terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{fidelity}} + \mathcal{L}_{\text{sparsity}}. \quad (11)$$

3.3. One-for-All reusing strategy

As for the pruned activations, we prefill them with the activations cached previously. A suitable reusing strategy can maximize the reduction of the caching error, thereby improving the pruning efficiency. Previous caching strategies operate locally within each block, maintaining a caching buffer for each block. However, this strategy overlooks redundancy across blocks. We observe in Figure 4a that activations across different blocks exhibit strong structural similarity, which motivates a one-for-all reusing strategy.

Concretely, instead of maintaining independent cache buffers for each block, SAG maintains a shared buffer τ for all blocks of the same type. When a block is executed, its activation updates the corresponding buffer; when a block is pruned, SAG reuses the cached activation from that buffer rather than recomputing the block. This design enables cross-block reuse in a zig-zag manner (See Fig. 4b), where an activation can be reused for another block in another timestep. Following the residual formulation in Eq. 4, the computation can be formulated as:

$$h_k^{[b/3]} = h_k^{[b/3]-1} + (1 - \mathcal{M}_k^b) \cdot d_k^b + \mathcal{M}_k^b \cdot \tau, \quad (12)$$

where d_k^b denotes the computation output of the b -th block at k -th step. The cache buffer τ is updated in tandem as:

$$\tau = (1 - \mathcal{M}_k^b) \cdot d_k^b + \mathcal{M}_k^b \cdot \tau. \quad (13)$$

Pipeline. We integrate the real-time diffusion pruner and one-for-all reusing strategy into the two-stage pipeline to boost the action diffusion collaboratively. As illustrated in Fig. 4c, at each rollout step, the real-time pruner predicts and decides which computations to prune before the action diffusion, conditioned on the specific observations at each iteration. During the action diffusion, the pruned computations are substituted by the cached activations with the one-for-all reusing strategy, which reuses activations across blocks and timesteps in a zig-zag manner. We adopt the one-for-all reusing strategy on both the training stage and the inference stage of the pruner. The training and inference procedures are summarized in Alg. 1 and Alg. 2.

4. Experiments

We first outline the experimental setup, covering models, benchmarks, and baselines in Sec. 4.1. Following that, we demonstrate the quantitative results in Sec. 4.2. We then present an ablation study of SAG in Sec. 4.3. Finally, we conduct qualitative experiments on the predicted sparsity pattern in Sec. 4.5. We also provide detailed training hyperparameters, hardware specifications, and network architecture details in Appendix A.3, and visualize the training process in Appendix A.4.

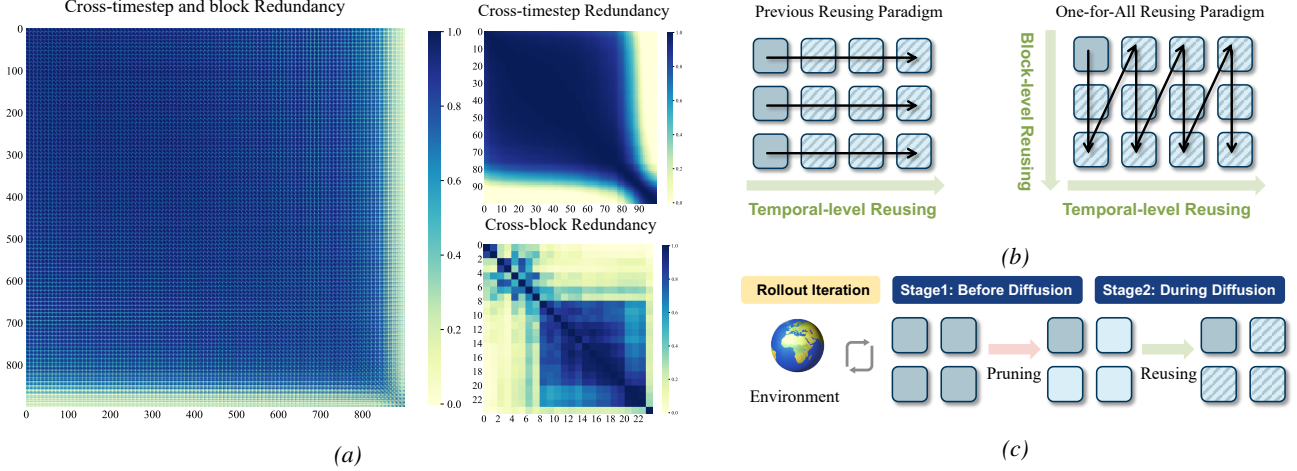


Figure 4. (a) Redundancy patterns at different levels, revealed by calculating the similarities between activations. The left subfigure shows the overall cross-timestep and cross-block redundancy in action generation, where each axis indexes unrolled (denoising step, block) pairs. The top-right subfigure illustrates the cross-timestep redundancy at the eighth cross-attention block, while the bottom-right subfigure shows the cross-block redundancy at timestep 30. See Appendix A.8 for more details. (b) Comparison of our one-for-all reusing paradigm with the previous block-wise reusing paradigm. The previous approach only reuses activations at the temporal level, whereas our paradigm enables both temporal and block-level reusing. (c) The prune-then-reuse pipeline of SAG, which consists of two stages: (1) *Prune* before the diffusion process to remove redundant computations, and (2) *Reuse* the remaining activations during the diffusion process.

Algorithm 1: Training

Require: Policy π_θ , Dataset $\mathcal{D}_{\text{ref}}$, Target Pruning Rate ρ , Factor β
Ensure: Trained Pruner G_ψ
 Initialize Pruner parameters ψ
 Freeze Policy parameters θ
while not converged **do**
 # Generate Pruning Mask
 Sample batch $(\mathbf{o}_t, \mathbf{a}_t^*)$ from $\mathcal{D}_{\text{ref}}$
 $\mathcal{M} = G_\psi(\mathbf{o}_t)$
 # Generate Action
 $\mathbf{a}_{\text{pred}} = \pi_\theta(\mathbf{o}_t, \mathcal{M})$
 $\mathcal{L}_{\text{fidelity}} = \|\pi'(\mathbf{o}_t, \mathcal{M}) - \mathbf{a}_t^*\|$ ▷ Eq. 9
 $\mathcal{L}_{\text{sparsity}} = \|\frac{1}{N} \sum_{k,b} \mathcal{M}_{k,b} - \rho\|$ ▷ Eq. 10
 $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{fidelity}} + \beta \mathcal{L}_{\text{sparsity}}$ ▷ Eq. 11
 Update $\psi \leftarrow \psi - \eta \nabla_\psi \mathcal{L}_{\text{total}}$
end while
Return G_ψ

Algorithm 2: Inference

Require: Observation $\mathbf{o}_t$, Pruner G_ψ , Denoiser f_θ , Steps K
Ensure: Action $\mathbf{a}_t$
 $\mathcal{M} \leftarrow G_\psi(\mathbf{o}_t)$ ▷ Eq. 5
 Initialize global cache $\tau \leftarrow \mathbf{0}$
 Sample noise $\mathbf{a}_t^K \sim \mathcal{N}(\mathbf{0}, I)$
for $k = K$ **to** 1 **do**
 $\mathbf{h} \leftarrow \text{Embed}(\mathbf{a}_t^k, k, \mathbf{o}_t)$
 for each block b in f_θ **do**
 if $\mathcal{M}_{k,b} == 1$ **then** ▷ Eq. 12
 $\mathbf{h} \leftarrow \mathbf{h} + \tau$
 else
 $\mathbf{d}_k^b \leftarrow \text{Block}_b(\mathbf{h})$
 $\tau \leftarrow \mathbf{d}_k^b$ ▷ Eq. 13
 $\mathbf{h} \leftarrow \mathbf{h} + \mathbf{d}_k^b$
 end if
 end for
 $\mathbf{a}_t^{k-1} \leftarrow \text{Decode}(\mathbf{h})$
end for
Return $\mathbf{a}_t^0$

4.1. Experimental Setup

Models and Benchmarks. Following the original settings in Diffusion Policy (Chi et al., 2023), we select the transformer-based Diffusion Policy (DP-T) for our evaluation. The pretrained checkpoints are from source¹. In simulation settings, we evaluate SAG on DP-T across different robot manipulation tasks with three fixed seeds: Lift, Can, Square, Transport, Tool hang, and Kitchen (Gupta et al., 2020). Demonstration data is sourced from proficient human (PH) and mixed proficient/non-proficient human (MH) teleoperation. More details can be found in Appendix A.1. In real-world settings, we use a Franka Research 3 robot with a front-view camera. We perform 3 tasks: 1) Stack

bowls, 2) Insert pen into the holder, 3) Plate the chocolate from the assembly line. For each task, 200 demonstrations are collected via teleoperation using Gelllo.

Baselines. We use the DP-T (Chi et al., 2023) model as our Full Computation baseline. For comparison, we benchmark against leading acceleration methods. We reproduce cache-based approaches, including Efficient-VLA (Yang et al., 2025) and BAC (Ji et al., 2025). We further include DDIM (Song et al., 2020), a sampling-efficient noise scheduler, and Falcon (Chen et al., 2025a), which accelerates denoising by reusing intermediate noise from previous actions. Finally, we evaluate the learning-based methods Streaming Diffusion Policy (SDP) (Høeg et al., 2024) and Consistency Policy (CP) (Prasad et al., 2024), which distills the generation into a few denoising steps via consistency distillation,

¹<https://diffusion-policy.cs.columbia.edu/data/experiments/>

Table 1. **Benchmark on Proficient Human (PH) demonstration data.** Success rates (%) and speedups are reported. SAG achieves more than a $3.4\times$ speedup across all tasks, with an average performance gain of 13%.

Method	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)				
	Lift	Can	Square	Transport	Tool
Full Computation	100 $\pm$ 0.0	99 $\pm$ 1.2	88 $\pm$ 7.0	78 $\pm$ 3.3	51 $\pm$ 5.9
DDIM	100 $\pm$ 0.0 (3.32 $\times$)	96 $\pm$ 2.8 (3.33 $\times$)	86 $\pm$ 4.9 (3.34 $\times$)	76 $\pm$ 5.7 (3.30 $\times$)	42 $\pm$ 4.6 (3.32 $\times$)
EfficientVLA	100 $\pm$ 0.0 (3.37 $\times$)	75 $\pm$ 2.1 (3.36 $\times$)	86 $\pm$ 3.4 (3.32 $\times$)	60 $\pm$ 4.2 (3.24 $\times$)	38 $\pm$ 2.7 (3.35 $\times$)
L2C	100 $\pm$ 0.0 (1.26 $\times$)	86 $\pm$ 1.8 (1.26 $\times$)	23 $\pm$ 4.1 (1.26 $\times$)	66 $\pm$ 3.0 (1.33 $\times$)	2 $\pm$ 0.5 (1.29 $\times$)
BAC	100 $\pm$ 0.0 (3.23 $\times$)	94 $\pm$ 1.5 (3.42 $\times$)	87 $\pm$ 2.9 (3.42 $\times$)	78 $\pm$ 3.6 (3.09 $\times$)	51 $\pm$ 2.8 (3.33 $\times$)
Falcon	100 $\pm$ 0.0 (1.81 $\times$)	85 $\pm$ 2.0 (1.13 $\times$)	60 $\pm$ 2.7 (1.22 $\times$)	50 $\pm$ 3.3 (2.85 $\times$)	42 $\pm$ 1.9 (1.17 $\times$)
SDP	100 $\pm$ 0.0 (1.88 $\times$)	96 $\pm$ 1.4 (1.70 $\times$)	85 $\pm$ 2.1 (1.70 $\times$)	72 $\pm$ 2.9 (1.67 $\times$)	17 $\pm$ 1.2 (1.77 $\times$)
CP	78 $\pm$ 3.3 (15.0 $\times$)	38 $\pm$ 2.8 (14.9 $\times$)	22 $\pm$ 4.5 (14.8 $\times$)	51 $\pm$ 3.1 (10.2 $\times$)	0 $\pm$ 0.0 (14.2 $\times$)
SAG	100 $\pm$ 0.0 (3.72 $\times$)	98 $\pm$ 1.6 (3.70 $\times$)	89 $\pm$ 2.5 (3.64 $\times$)	85 $\pm$ 3.3 (3.44 $\times$)	50 $\pm$ 2.8 (3.65 $\times$)

Table 2. **Benchmark on Mixed Human (MH) demonstration data.** SAG achieves over a $3.7\times$ speedup across all tasks, with an average performance gain of 29%.

Method	Success Rate (% , $\uparrow$) / Speedup ($\uparrow$)			
	Lift	Can	Square	Transport
Full Computation	100 $\pm$ 0.0	93 $\pm$ 6.5	76 $\pm$ 4.3	54 $\pm$ 5.2
DDIM	100 $\pm$ 0.0 (3.31 $\times$)	92 $\pm$ 4.9 (3.33 $\times$)	78 $\pm$ 2.8 (3.32 $\times$)	51 $\pm$ 3.8 (3.30 $\times$)
EfficientVLA	100 $\pm$ 0.0 (3.33 $\times$)	75 $\pm$ 2.1 (3.34 $\times$)	52 $\pm$ 3.0 (3.33 $\times$)	0 $\pm$ 0.0 (3.50 $\times$)
L2C	100 $\pm$ 0.0 (1.26 $\times$)	0 $\pm$ 0.0 (1.26 $\times$)	53 $\pm$ 1.9 (1.26 $\times$)	46 $\pm$ 2.3 (1.28 $\times$)
BAC	100 $\pm$ 0.0 (3.27 $\times$)	93 $\pm$ 1.6 (3.43 $\times$)	78 $\pm$ 2.8 (3.36 $\times$)	29 $\pm$ 3.4 (3.45 $\times$)
Falcon	100 $\pm$ 0.0 (1.85 $\times$)	84 $\pm$ 2.0 (1.38 $\times$)	40 $\pm$ 2.9 (1.54 $\times$)	27 $\pm$ 3.1 (2.68 $\times$)
SDP	100 $\pm$ 0.0 (1.69 $\times$)	94 $\pm$ 1.4 (1.70 $\times$)	77 $\pm$ 2.2 (1.69 $\times$)	52 $\pm$ 3.0 (1.83 $\times$)
CP	98 $\pm$ 1.0 (15.35 $\times$)	15 $\pm$ 2.5 (15.1 $\times$)	30 $\pm$ 3.1 (15.4 $\times$)	14 $\pm$ 2.0 (11.5 $\times$)
SAG	100 $\pm$ 0.0 (3.75 $\times$)	94 $\pm$ 5.7 (3.76 $\times$)	79 $\pm$ 3.4 (3.72 $\times$)	50 $\pm$ 1.6 (3.84 $\times$)

Table 3. **Benchmark on multi-stage task (Kitchen).** In Kitchen task, p_x is the frequency of interacting with x or more objects, including an openable microwave, four turnable oven burners, an oven light switch, and a freely movable kettle. SAG achieves a $4.03\times$ speedup while maintaining comparable performance.

Method	Success Rate (% , $\uparrow$)				Speedup
	Kit $_{p1}$	Kit $_{p2}$	Kit $_{p3}$	Kit $_{p4}$	
Full Computation	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	99 $\pm$ 0.6	–
DDIM	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	99 $\pm$ 0.8	3.37 $\times$
EfficientVLA	20 $\pm$ 2.3	2 $\pm$ 0.8	0 $\pm$ 0.0	0 $\pm$ 0.0	3.71 $\times$
L2C	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	97 $\pm$ 1.2	1.28 $\times$
BAC	100 $\pm$ 0.0	100 $\pm$ 0.0	97 $\pm$ 1.6	90 $\pm$ 2.9	3.66 $\times$
Falcon	100 $\pm$ 0.0	100 $\pm$ 0.0	99 $\pm$ 0.6	99 $\pm$ 0.6	3.01 $\times$
SDP	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	99 $\pm$ 0.6	1.63 $\times$
CP	76 $\pm$ 2.5	63 $\pm$ 4.3	46 $\pm$ 4.5	12 $\pm$ 6.8	31.4 $\times$
SAG	100 $\pm$ 0.0	100 $\pm$ 0.0	100 $\pm$ 0.0	99 $\pm$ 0.6	4.03 $\times$

and L2C (Ma et al., 2024a), adapted from image generation to our action-generation setting.

4.2. Quantitative Results

To show the advantage of SAG, we compare our method with several SOTA acceleration methods (Yang et al., 2025;

Ma et al., 2024a; Ji et al., 2025). We set the cache updating interval $\mathcal{N}$ as 7 for Efficient VLA. For L2C, we adopt the same training configs as SAG for fair comparison. We set the number of cache update steps $\mathcal{S}$ as 10 and the number of selected upstream blocks k as 5 for BAC, following the original paper. We set the inference steps K as 30 for DDIM and 3 for Consistency Policy. For Falcon, we follow the original setup (Chen et al., 2025a).

Overall Results. As shown in Tables 1, 2, and 3, SAG demonstrates a superior trade-off between performance and efficiency across all the simulation benchmarks. On the PH and MH datasets, SAG improves the average success rates to 84% and 81%, respectively. SAG also achieves substantial speedups of $3.63\times$ and $3.77\times$ with an extreme pruning rate exceeding 90%. The advantage is most pronounced on the multi-stage Kitchen task, where SAG achieves 100% success rate while delivering up to $4\times$ speedup. The latency is detailed in Appendix A.6. In real-world evaluations, Table 4 demonstrates that SAG maintains high inference frequencies (44.3–46.1 Hz) while achieving competitive success rates across all tasks. Fig. 5 further shows that SAG raises Assembly Line success from 26% to 58%, validating its superiority in tasks requiring high-fidelity responsiveness.

Table 7. Success Rates under Different Pruning Rates.

Pruning Rate	Can _{ph}	Lift _{ph}	Sq _{ph}	Tool _{ph}	Trans _{ph}	Can _{mh}	Lift _{mh}	Sq _{mh}	Trans _{mh}	Kitchen	Avg
80%	96	100	92	60	82	98	100	76	44	99	85
91%	98	100	89	50	85	94	100	79	50	100	85
97%	2	70	70	54	70	66	48	34	0	98	51

Table 4. Real-world experiments.

Method	Success Rate (% , $\uparrow$) / Frequency (Hz, $\uparrow$)		
	Stack Bowls	Insert Pen	Assembly Line
Full Computation	82 (7.1)	28 (6.8)	26 (7.4)
SAG	78 (46.1)	30 (45.6)	58 (44.3)

Comparison with SOTA baselines. Notably, the baselines fail to maintain comparable performance with similar acceleration gains. EfficientVLA uses a fixed caching schedule for any input, particularly on the Kitchen task, where its success rate falls to 3%. L2C’s rigid policy of caching every two steps inherently caps its acceleration, resulting in a modest $1.28\times$ speedup. BAC cannot adapt to evolving rollout dynamics and remains tied to a block-wise caching paradigm, performing poorly on complex tasks such as transport. The excessive compression rate seriously impairs the performance of CP, while Falcon and SDP achieve sub-optimal speedup due to coarse-grained pruning strategies. In contrast, SAG achieves environment-aware, real-time pruning to adapt to the rollout dynamics and achieves a global prune-then-reuse pipeline to preserve fidelity of generated actions under extremely high sparsity.

Compatibility with Efficient Samplers. Since SAG prunes computations inside each denoising step, it is complementary to sampler-level acceleration. To verify this, we pair SAG with DPM-Solver++ in Table 5. DPM10 reduces the number of solver steps but causes substantial performance degradation on several tasks. In contrast, applying SAG to the stronger DPM50 schedule preserves the DPM50 success rates across most tasks and improves Square_{mh}, indicating that SAG is not merely mimicking a shorter denoising trajectory.

 Table 5. **Compatibility with DPM-Solver++.** Success rates (%) are reported.

Method	Can _{ph}	Sq _{ph}	Lift _{ph}	Can _{mh}	Sq _{mh}	Tool
DPM10	74	48	2	52	56	6
DPM50	92	84	100	86	72	46
DPM50+SAG	92	84	100	86	78	46

4.3. Ablation Study

To verify the effectiveness of the key design choices of SAG, we conduct a series of ablation experiments, with results summarized in Table 6.

Effectiveness of the Real-time Diffusion Pruner. For comparison, we adopt the diffusion pruner to generate a fixed caching schedule and substitute the pruner with this schedule throughout the entire rollout. The results show that the substitution leads to a noticeable drop in performance, with the average success rate decreasing from 89% to 83% on the MH and Kitchen tasks. This degradation highlights that achieving environmental awareness and real-time pruning is the key to efficient action generation without performance degradation.

Effectiveness of the One-for-All Reusing Strategy. We replace the one-for-all reusing strategy with the widely adopted block-wise reusing strategy to isolate its impact on the overall performance. The results show that the reusing strategy improves the performance significantly by 12% on average. This demonstrates that the proposed strategy reduces both the cross-block and cross-timestep redundancy.

Effectiveness of the Global Sparsity Loss. We assess the global sparsity loss by substituting it with a block-wise loss, which assigns a uniform computational budget to every block. The removal of global sparsity loss leads to a catastrophic collapse in performance across all tasks, with the average success rate dropping by 37% on PH tasks and 32% on MH tasks. Notably, performance on several tasks, such as Square and Transport_{mh}, drops to 0%. This underscores that allocating as many computational resources as possible to important blocks is critical to achieving extreme sparsity.

4.4. Sensitivity Analysis

We study the sensitivity of SAG to different target pruning rates. As shown in Table 7, SAG keeps an 85% average success rate at both 80% and 91% pruning, showing that redundant computations can be removed aggressively without disrupting the denoising trajectory. Since 91% pruning preserves the same average success rate with a smaller compute budget, we use it as the default setting. When the rate increases to 97%, the average success rate drops to 51%, with failures mainly appearing on contact-sensitive Can and Transport variants while Kitchen remains robust. This suggests that extreme pruning primarily removes computations needed for sustained fine-grained corrections. We provide a compute-matched comparison with Consistency Policy in Appendix A.5.

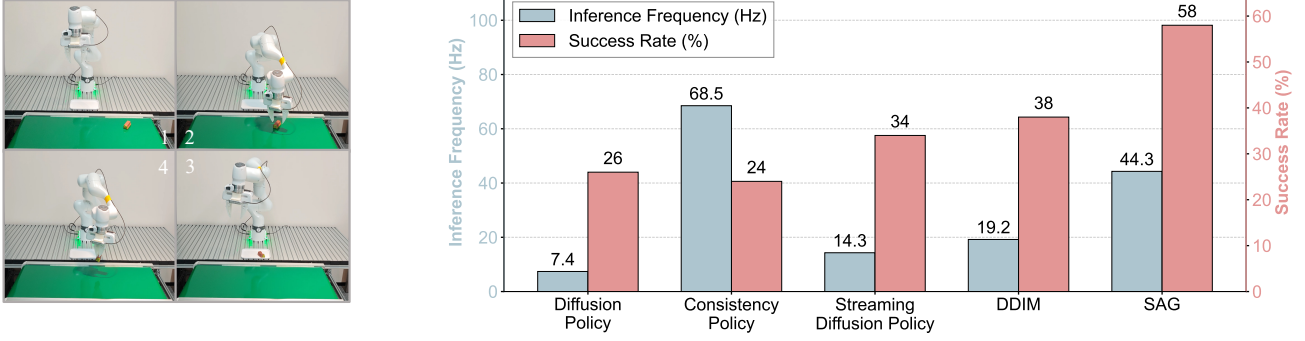


Figure 5. Left: Assembly Line Task Setup. Right: Real-world evaluation results, measured by inference frequency and success rate.

Table 6. **Ablation Study.** We evaluate three key design choices of SAG: the real-time diffusion pruner, the one-for-all reusing strategy, and the global sparsity loss. Experiments show that removing any component results in a significant drop in the average success rate.

Method	Success Rate (% , $\uparrow$)										
	Lift _{ph}	Can _{ph}	Sq _{ph}	Trans _{ph}	Tool	Lift _{mh}	Can _{mh}	Sq _{mh}	Trans _{mh}	Kit	Avg
w/o Real-time Diffusion Pruner	100	92	90	80	48	100	94	76	46	99	83
w/o One-for-All Reusing Strategy	100	94	94	0	36	100	94	80	48	97	74
w/o Global Sparsity Loss	100	86	0	16	40	100	90	0	0	93	53
SAG	100	94	94	84	54	100	96	86	62	100	87

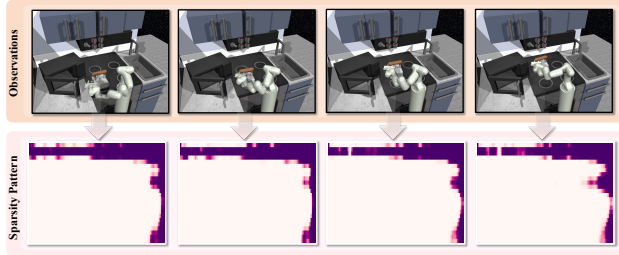


Figure 6. **Predicted sparsity patterns across rollout iterations when the robot moves the kettle.** A coordinate point (x,y) in the sparsity pattern figures represents the computation of the y-th block at the x-th timestep. Lighter colors indicate higher sparsity, and the changing masks show how SAG reallocates computation as observations evolve during rollout.

4.5. Qualitative Results.

We visualize the sparsity patterns predicted by the pruner while the robot moves a kettle in Figure 6. We observe that most computations are highly sparse, as shown by the large white regions. Moreover, the pattern shifts with each observation, demonstrating the pruner’s ability to adapt to the evolving rollout dynamics. Interestingly, the computations become dense in the top and right areas, indicating that the upper blocks and later denoising steps exhibit more significant changes in activation patterns. We also present details of sparsity patterns for different tasks in Appendix A.7.

5. Limitations

Although SAG demonstrates strong efficiency across the evaluated benchmarks, its current scope is still bounded by the policy families, robot platforms, and manipulation tasks studied in this paper. Extending the same idea to larger generalist policies, more diverse hardware, longer-horizon tasks, and broader deployment conditions remains an important direction. SAG also uses a fixed target compute budget and trains the pruner from representative reference data; future work may explore adaptive budgets and more data-efficient pruner training for open-world robot deployment.

6. Conclusion

Real-time visuomotor control requires action generation to be both fast and responsive to changing robot-environment dynamics. This motivates real-time pruning, which allocates computation according to the current observation and rollout state rather than a fixed schedule. In this work, we propose Sparse ActionGen (SAG) for sparse action diffusion. SAG boosts action generation with two key designs. First, SAG designs a real-time pruner that adapts caching schedules to the rollout dynamics during robot–environment interaction. Second, SAG develops a global prune-then-reuse pipeline that enables both global allocation of computational resources and global reuse of activations across blocks and timesteps. Extensive real-robot and simulation experiments demonstrate SAG’s superior efficiency and effectiveness.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (Grant No. 92467204, 62472249, and 62402264), and the Shenzhen Science and Technology Program (Grant No. JCYJ20220818101014030 and KJZD20240903102300001).

Impact Statement

In this work, all experiments are conducted on publicly available robotic simulation datasets, including RoboMimic, and Franka Kitchen. The demonstration data were originally collected under published protocols via teleoperation or scripted policies, and no new human subjects or personal data were involved. Our study does not raise concerns regarding privacy, discrimination, or bias, since the tasks are synthetic robotic manipulation environments. The proposed method, Sparse ActionGen, focuses solely on improving computational efficiency of diffusion-based robotic policies, and we identify no risks of rights infringement, legal non-compliance, or unethical use.

References

- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. *pi_0: A vision-language-action flow model for general robot control*. *arXiv preprint arXiv:2410.24164*, 2024.
- Chen, H., Liu, M., Ma, C., Ma, X., Ma, Z., Wu, H., Chen, Y., Zhong, Y., Wang, M., Li, Q., and Yang, Y. Falcon: Fast visuomotor policies via partial denoising. *arXiv preprint arXiv:2503.00339*, 2025a. URL <https://arxiv.org/abs/2503.00339>.
- Chen, P., Shen, M., Ye, P., Cao, J., Tu, C., Bouganis, C.-S., Zhao, Y., and Chen, T. δ -dit: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.
- Chen, Z., Yuan, X., Mu, T., and Su, H. Responsive noise-relaying diffusion policy: Responsive and efficient visuomotor control. *arXiv preprint arXiv:2502.12724*, 2025b.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, pp. 02783649241273668, 2023.
- Gupta, A., Kumar, V., Lynch, C., Levine, S., and Hausman, K. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.
- Gupta, A., Kumar, V., Lynch, C., Levine, S., and Hausman, K. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. In *Conference on Robot Learning*, pp. 1025–1037. PMLR, 2020.
- Hou, Z., Zhang, T., Xiong, Y., Duan, H., Pu, H., Tong, R., Zhao, C., Zhu, X., Qiao, Y., Dai, J., et al. Dita: Scaling diffusion transformer for generalist vision-language-action policy. *arXiv preprint arXiv:2503.19757*, 2025.
- Høeg, S. H., Du, Y., and Egeland, O. Streaming diffusion policy: Fast policy synthesis with variable noise diffusion models. *arXiv preprint arXiv:2406.04806*, 2024. URL <https://arxiv.org/abs/2406.04806>.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- Ji, K., Meng, Y., Cui, H., Li, Y., Hua, S., Chen, L., and Wang, Z. Block-wise adaptive caching for accelerating diffusion policy. *arXiv preprint arXiv:2506.13456*, 2025.
- Ji, K., Meng, Y., Zhou, J., Li, Y., Tang, C., and Wang, Z. Test-time sparsity for extreme fast action diffusion, 2026. URL <https://arxiv.org/abs/2605.13316>.
- Jia, B., Ding, P., Cui, C., Sun, M., Qian, P., Huang, S., Fan, Z., and Wang, D. Score and distribution matching policy: Advanced accelerated visuomotor policies via matched distillation. *arXiv preprint arXiv:2412.09265*, 2024.
- Li, Y., Meng, Y., Sun, Z., Ji, K., Tang, C., Fan, J., Ma, X., Xia, S., Wang, Z., and Zhu, W. Sp-vla: A joint model scheduling and token pruning approach for vla model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
- Liu, F., Zhang, S., Wang, X., Wei, Y., Qiu, H., Zhao, Y., Zhang, Y., Ye, Q., and Wan, F. Timestep embedding tells: It’s time to cache for video diffusion model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 7353–7363, 2025a.
- Liu, J., Chen, H., An, P., Liu, Z., Zhang, R., Gu, C., Li, X., Guo, Z., Chen, S., Liu, M., et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025b.
- Liu, S., Wu, L., Li, B., Tan, H., Chen, H., Wang, Z., Xu, K., Su, H., and Zhu, J. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- Liu, Y., Liu, Y., Meng, Y., Zhang, J., Zhou, Y., Li, Y., Jiang, J., Ji, K., Ge, S., Wang, Z., et al. Spatial policy: Guiding visuomotor robotic manipulation with spatial-aware modeling and reasoning. *arXiv preprint arXiv:2508.15874*, 2025c.

- Ma, X., Fang, G., Bi Mi, M., and Wang, X. Learning-to-cache: Accelerating diffusion transformer via layer caching. *Advances in Neural Information Processing Systems*, 37:133282–133304, 2024a.
- Ma, X., Fang, G., and Wang, X. Deepcache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15762–15772. Placeholder Publisher, 2024b.
- Mandlekar, A., Xu, D., Wong, J., Nasiriany, S., Wang, C., Kulkarni, R., Fei-Fei, L., Savarese, S., Zhu, Y., and Martín-Martín, R. What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2021.
- Prasad, A., Lin, K., Wu, J., Zhou, L., and Bohg, J. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.
- Selvaraju, P., Ding, T., Chen, T., Zharkov, I., and Liang, L. Fora: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.
- Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- Wang, Z., Li, Z., Mandlekar, A., Xu, Z., Fan, J., Narang, Y., Fan, L., Zhu, Y., Balaji, Y., Zhou, M., et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*, 2024.
- Wen, J., Zhu, Y., Li, J., Tang, Z., Shen, C., and Feng, F. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025. Submitted to Some Conference, unpublished.
- Wimbauer, F., Wu, B., Schoenfeld, E., Dai, X., Hou, J., He, Z., Sanakoyeu, A., Zhang, P., Tsai, S., Kohler, J., et al. Cache me if you can: Accelerating diffusion models through block caching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6211–6220, 2024.
- Yang, Y., Wang, Y., Wen, Z., Zhongwei, L., Zou, C., Zhang, Z., Wen, C., and Zhang, L. Efficientvla: Training-free acceleration and compression for vision-language-action models. *arXiv preprint arXiv:2506.10100*, 2025.
- Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., and Xu, H. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.
- Zhou, H., Blessing, D., Li, G., Celik, O., Jia, X., Neumann, G., and Lioutikov, R. Variational distillation of diffusion policies into mixture of experts. *Advances in Neural Information Processing Systems*, 37:12739–12766, 2024.
- Zhu, H., Tang, D., Liu, J., Lu, M., Zheng, J., Peng, J., Li, D., Wang, Y., Jiang, F., Tian, L., et al. Dip-go: A diffusion pruner via few-step gradient optimization. *Advances in Neural Information Processing Systems*, 37:92581–92604, 2024.
- Zou, C., Liu, X., Liu, T., Huang, S., and Zhang, L. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations (Poster Track)*, 2025. URL <https://openreview.net/forum?id=yYZbZGo4ei>.

A. Appendix

A.1. More details on the Benchmark

A.1.1. DATASETS

Franka Kitchen. This dataset originates from the Relay Policy Learning framework proposed by Gupta et al. (Gupta et al., 2019), featuring 566 VR tele-operated demonstrations of multi-step manipulation tasks in a simulated kitchen using a 9-DoF Franka Panda arm. The goal is to execute as many demonstrated tasks as possible, regardless of order, showcasing both short-horizon and long-horizon multimodality (Chi et al., 2023).

RoboMimic. This dataset, introduced by Mandlekar et al. (2021), covers five manipulation tasks. Each task includes a Proficient-Human (PH) teleoperated demonstration set, and four of the tasks additionally offer Mixed-Human (MH) sets combining proficient and non-proficient operators (9 variants in total). The PH data were recorded by a single operator via the RoboTurk platform, whereas the MH sets were collected from six different operators using the same system.

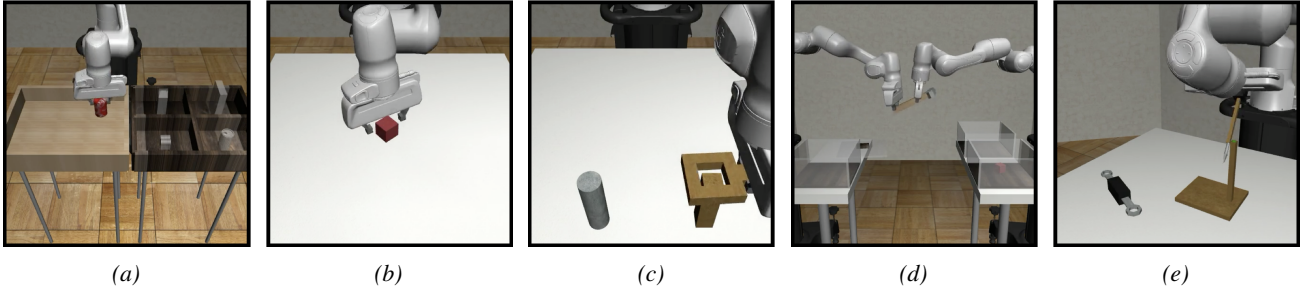


Figure 7. Visualizations of different tasks. (a) Can (b) Lift (c) Square (d) Transport (e) Tool_Hang.

Fig. 7 illustrates the five subtasks in the RoboMimic Image dataset. Below we describe each subtask.

- **Can** (Fig. 7a): The robot must grasp a cylinder-shaped object and placing it into a bin. This subtask tests precise grasp planning and fingertip control under varying object poses (Mandlekar et al., 2021).
- **Lift** (Fig. 7b): The manipulator picks up a heavier, irregularly shaped object (e.g. a small box) and raises it to a designated height. It evaluates the policy’s ability to modulate grip force and maintain stable trajectories (Mandlekar et al., 2021).
- **Square** (Fig. 7c): The agent must push or slide an object so that its center follows a square-shaped path on the table. This challenges both straight-line control and precise cornering maneuvers (Mandlekar et al., 2021).
- **Transport** (Fig. 7d): The agent must learn bimanual maneuvers to transfer a hammer from a closed container on a shelf to the target bin on another shelf. It tests coordinated lifting and translational motion under variable loads (Mandlekar et al., 2021).
- **Tool_Hang** (Fig. 7e): The robot arm must learn high-precision manipulation behaviors to assemble a frame by inserting a hook into a narrow base. This requires fine-tuned wrist orientation and insertion accuracy (Mandlekar et al., 2021).

A.1.2. PRE-TRAINED CHECKPOINTS

We use the pre-trained checkpoints of `diffusion_policy_transformer` model provided by Diffusion Policy (Chi et al., 2023), where checkpoints of image-based tasks are stored under link² and those of multi-stage tasks are stored under link³.

²<https://diffusion-policy.cs.columbia.edu/data/experiments/image/>

³https://diffusion-policy.cs.columbia.edu/data/experiments/low_dim/

A.2. Pruning rate

To further verify the stability of SAG’s acceleration effect, we conduct an evaluation of its real-time pruning behavior during inference under a target pruning rate of 91%. The experimental results demonstrate that SAG maintains a highly stable real-time pruning process, with the observed real-time pruning rate deviating by no more than 1.6% from the target.

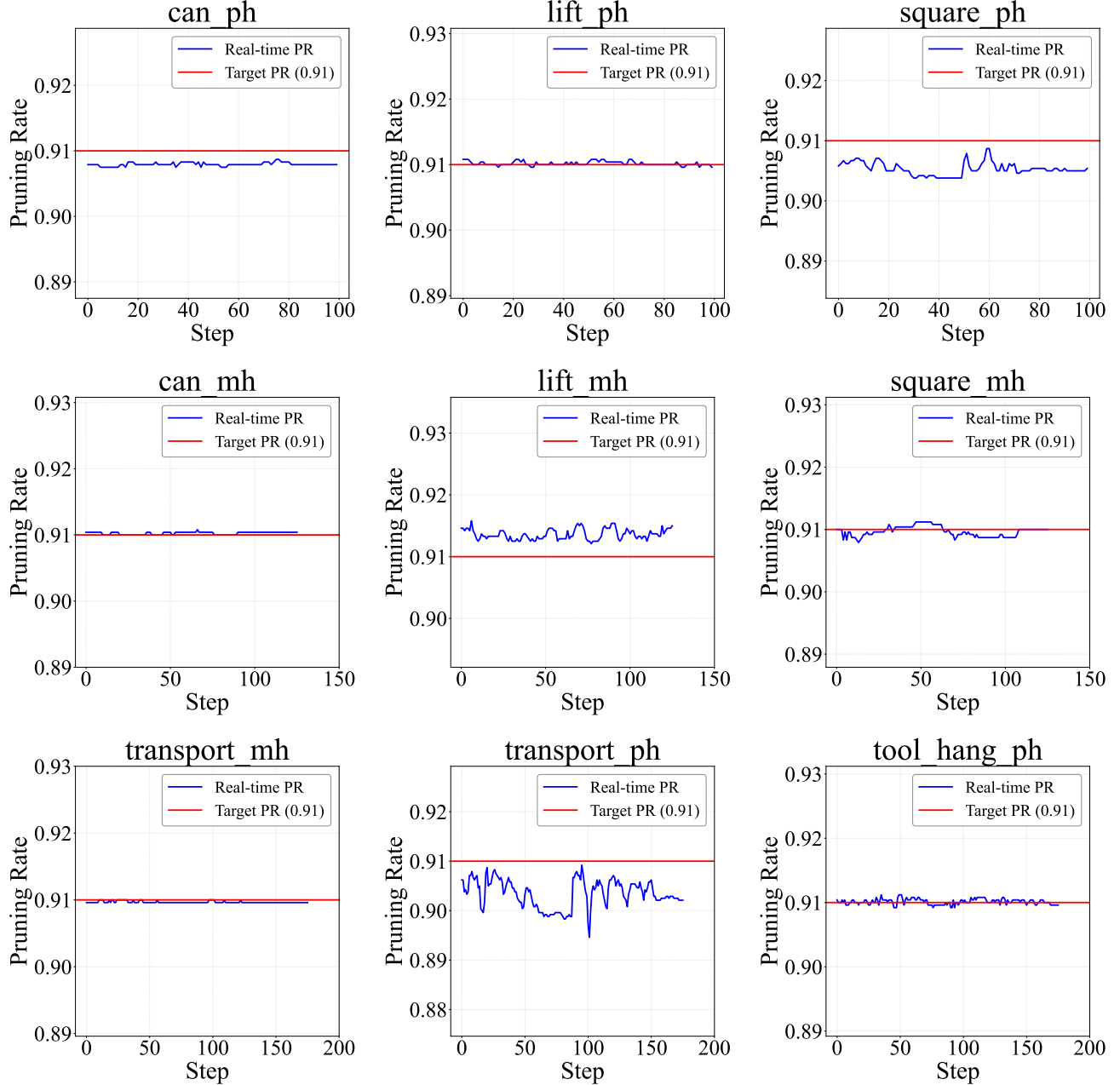


Figure 8. Real-time pruning rate throughout the entire rollout.

A.3. Implementation Details.

We conduct the simulation experiments on NVIDIA Tesla V100S 32G GPU, equipped with Intel(R) Xeon(R) Silver 4210 CPU. For real-world experiments, we conduct the experiments on Franka Research 3 robot arm with NVIDIA GeForce RTX 4090D 48G. The proposed pruner introduces a hyperparameter pruning rate ρ , we empirically set it as 91%. The pruner network is trained for 30 epochs with a batch size of 32 and a learning rate of $1e^{-4}$. We use a training set of 5,600 samples

and a validation set of 640 samples for each simulation task and 200 trajectories collected by Gello for the real-world task. We adopt a linear warmup of 4 steps and apply module-specific weight decay, set as $1e^{-4}$ for the observation encoder, $1e^{-3}$ for the transformer encoder, and $1e^{-5}$ for the three-layer MLP.

A.4. Loss Curves

We present training curves of SAG under target pruning ratios of 80%, 85%, and 91%. As shown in Fig. 9 and Fig. 10, the training loss decreases smoothly and converges to low values, while the validation loss remains stable with only minor fluctuations. These results provide evidence that SAG can achieve significant inference acceleration without incurring measurable degradation in predictive performance, thereby exhibiting robustness and reliability across a broad spectrum of pruning intensities.

A.5. Compute-matched Budget Comparison

We compare CP and SAG at matched effective inference budgets. CP is evaluated with 3, 9, and 20 chained denoising steps, while SAG uses 97%, 91%, and 80% target pruning rates on the 100-step DDPM schedule, corresponding to approximately the same effective number of full-model steps. As shown in Table 8, SAG achieves substantially higher average success rates at the 9-step and 20-step budgets, indicating that sparse within-step computation preserves policy fidelity better than aggressively shortening the denoising trajectory.

Table 8. Compute-matched summary. PH and MH denote average success rates over the corresponding benchmark groups; Avg averages PH, MH, and Kitchen tasks.

Budget	Method	PH	MH	Kit.	Avg
3	CP	38	43	49	41
	SAG	53	37	98	51
9	CP	35	41	60	40
	SAG	84	81	100	84
20	CP	36	38	57	39
	SAG	86	80	99	85

A.6. Detailed Latency on Simulation Experiments

We present the detailed latency of SAG on the simulation benchmarks in Table 9. The speedup is calculated by dividing the ratio of the wall-clock time after acceleration by the original time.

Table 9. Inference Latency.

Method	Latency (s, ↓)					Average
	Lift	Can	Square	Transport	Tool	
Full Computation	1.55	1.54	1.56	1.61	1.56	1.66
DDIM	0.47	0.46	0.47	0.49	0.47	0.47
EfficientVLA	0.46	0.46	0.47	0.50	0.47	0.50
L2C	1.23	1.22	1.24	1.21	1.21	1.29
BAC	0.48	0.45	0.46	0.52	0.47	0.50
Falcon	0.86	1.36	1.28	0.57	1.33	1.16
SDP	0.82	0.91	1.92	0.96	0.88	0.95
CP	0.10	0.10	0.11	0.16	0.11	0.12
SAG	0.42	0.42	0.43	0.47	0.43	0.46

A.7. More details on Sparsity Pattern Visualization

In this section, we present the predicted sparsity patterns by the pruner across different tasks. As shown in Fig. 11, the patterns differ for different tasks, showcasing that SAG can capture task-specific sparsity patterns, thereby achieving environment-aware adaptivity.

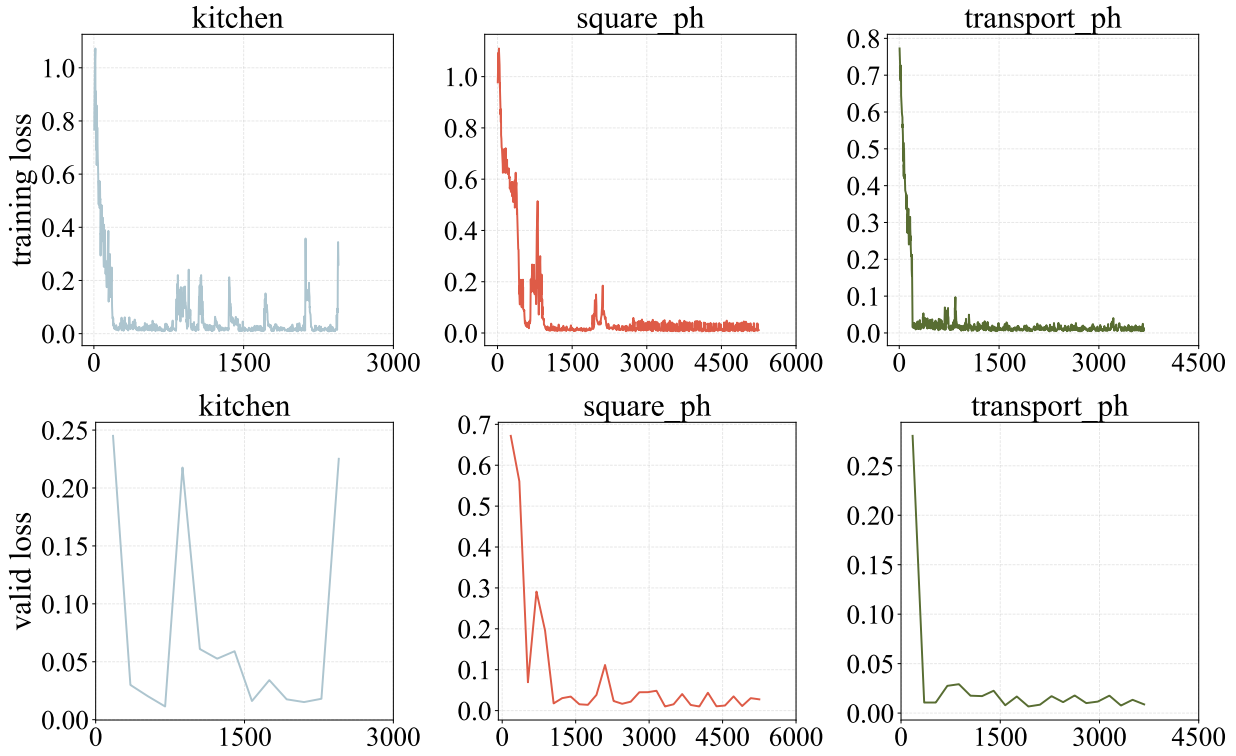
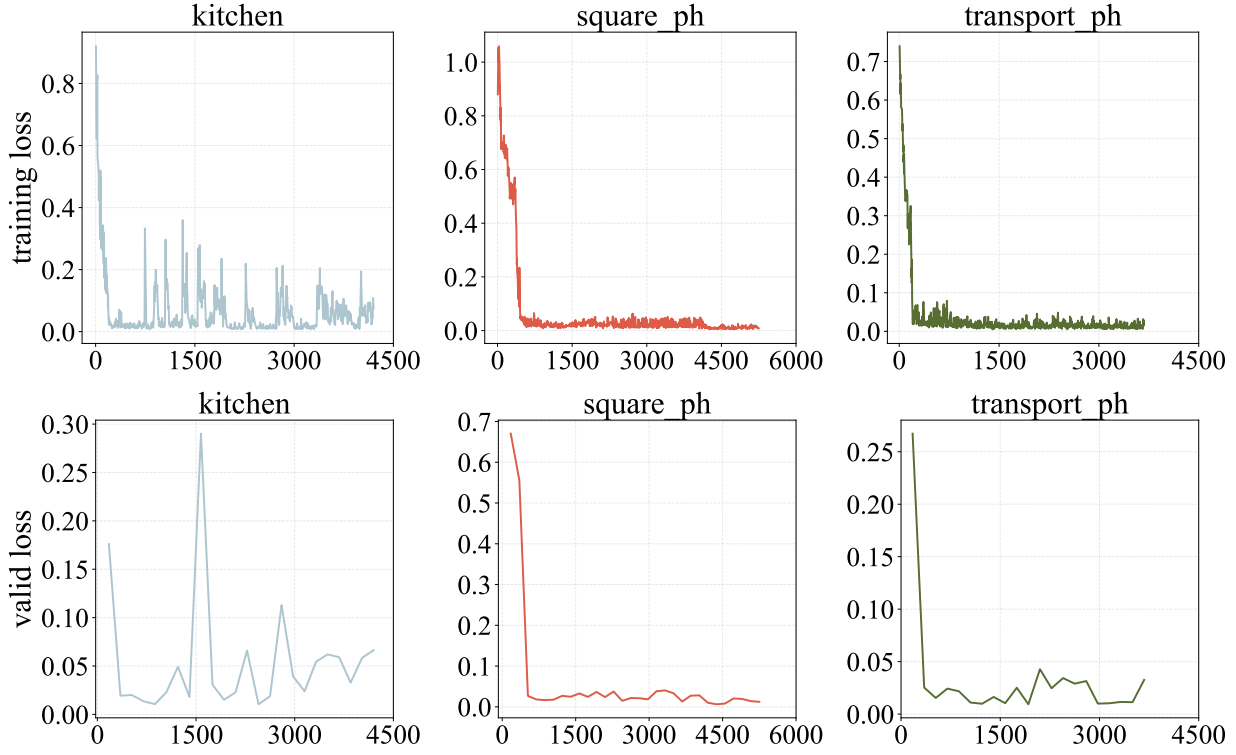


Figure 9. Loss curves under different target pruning rates (part 1).

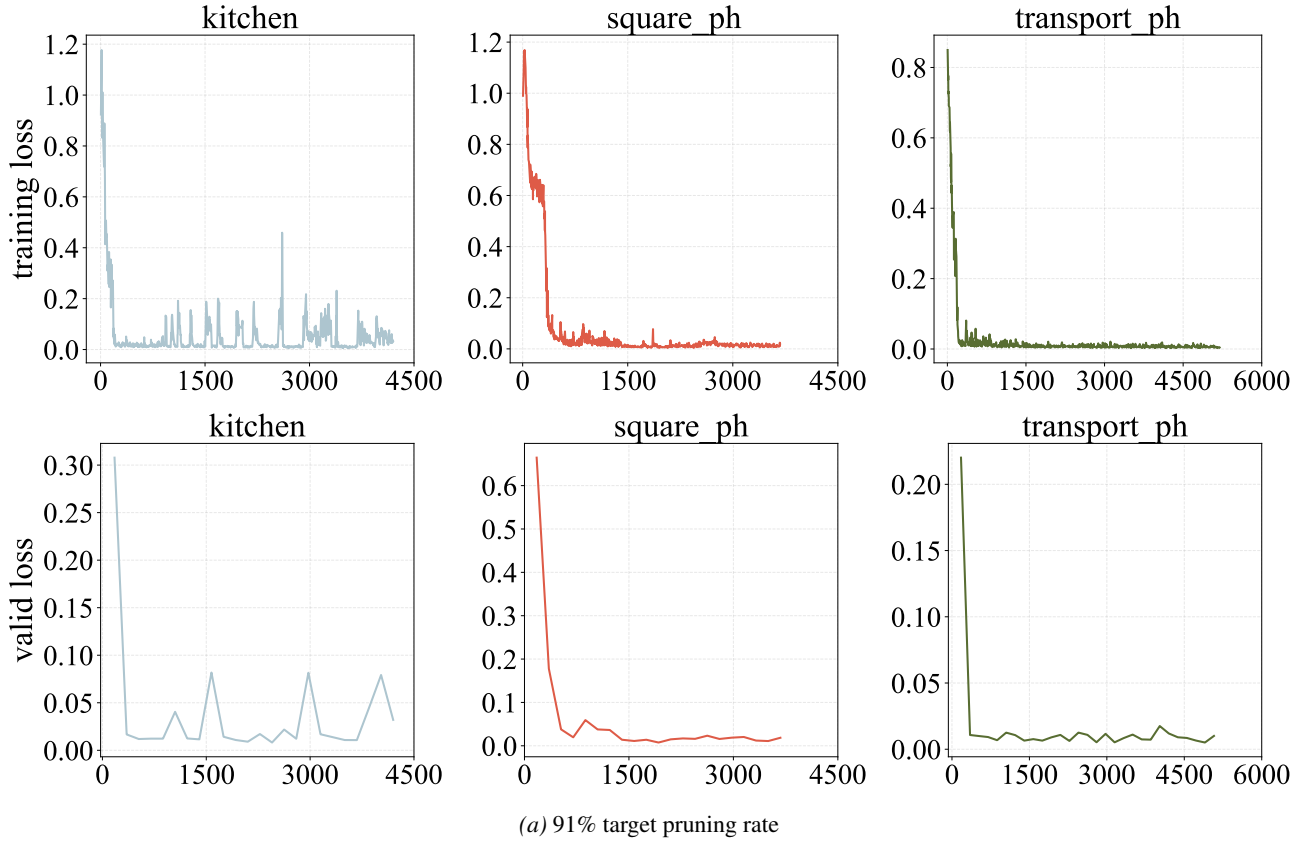


Figure 10. Loss curves under different target pruning rates (part 2).

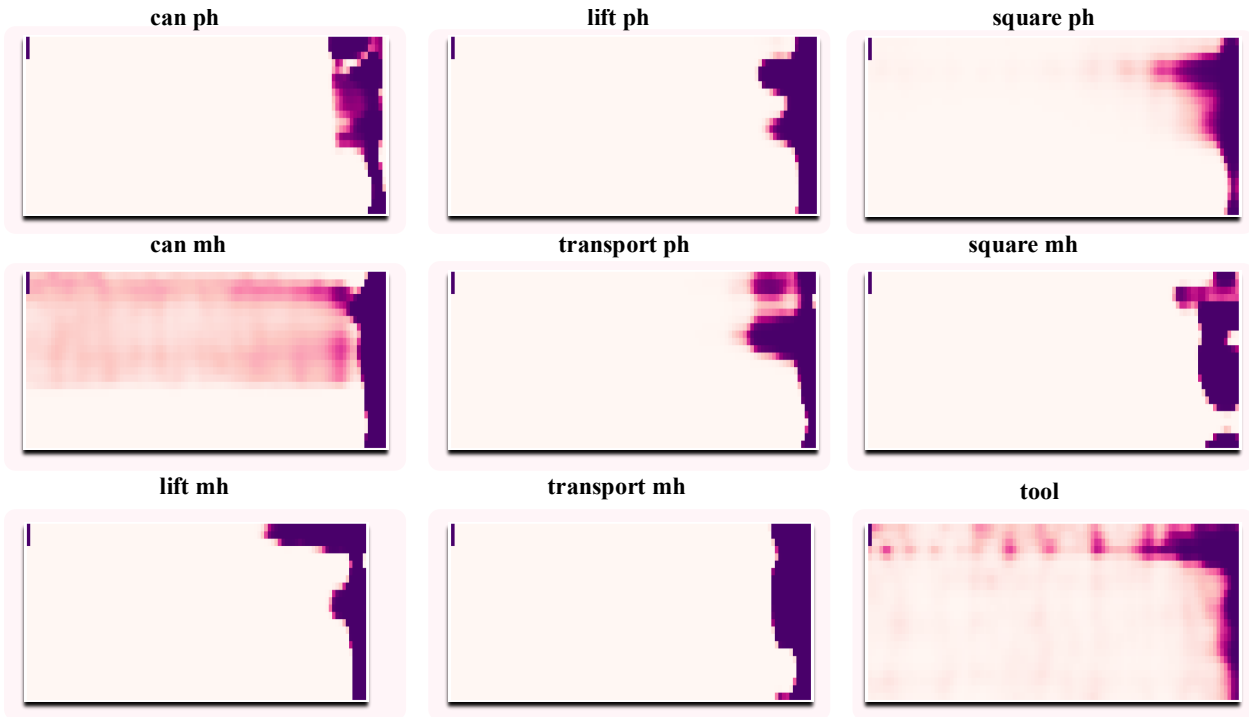


Figure 11. Predicted sparsity patterns for different tasks.

A.8. More details on Redundancy Visualization

To investigate the block-level redundancy within the DiT module of the diffusion policy, we conduct a comprehensive cosine-similarity analysis across every denoising step. In particular, for each individual denoising step, we examine all transformer layers of the DiT module—specifically the eight layers of self-attention, cross-attention, and feed-forward (FFN) blocks.

The visualization results reveal that every step consistently contains a substantial proportion of redundant computations within these components, and that the pairwise similarity patterns remain highly consistent across the entire denoising process.

Such pronounced cross-step and intra-layer correlations indicate that many block operations contribute little new information, suggesting that the effective representational capacity of the module is considerably lower than its nominal parameter count implies.

A.9. More Details on Sinusoidal Positional Encoding

Since step and block identifiers serve as the primary inputs to SAG, their representation plays a critical role in determining pruning quality. To examine this effect, we conduct experiments under a target pruning rate of 91% comparing sinusoidal positional encoding with a straightforward learned embedding (nn.Embedding) scheme. The results clearly show that sinusoidal positional encoding delivers substantially superior performance, highlighting the importance of well-structured positional information for stable and accurate pruning.

A.10. Block Participation

We measure how often each transformer sub-block is computed rather than pruned during closed-loop rollouts. Although the average active ratio is close to the target budget, participation is highly non-uniform across blocks and tasks, indicating that SAG reallocates computation instead of pruning every block uniformly.

A.11. Observation Robustness

We test whether observation shifts introduce additional failure modes for SAG. In real-world experiments, SAG is comparable to Full Computation under hand occlusion and laser distraction, while both methods fail under low light. In simulation, performance gaps between Full Computation and SAG remain small across four visual perturbations, suggesting that severe observation shifts mainly challenge the underlying policy rather than the pruning mechanism itself.

Table 10. Simulation robustness averaged over the 9 image-based benchmark tasks.

Model	Clean	Light Drop	Bg Tint	Distractor	Occlusion
Full Computation	81	33	54	38	28
SAG	83	34	48	36	27

Table 11. Real-robot robustness. Each cell reports successes over 20 trials.

Task	Hand FC	Hand SAG	Laser FC	Laser SAG	Low Light FC	Low Light SAG
Stack Bowls	13/20	12/20	14/20	15/20	0/20	0/20
Assembly Line	4/20	10/20	4/20	11/20	0/20	0/20

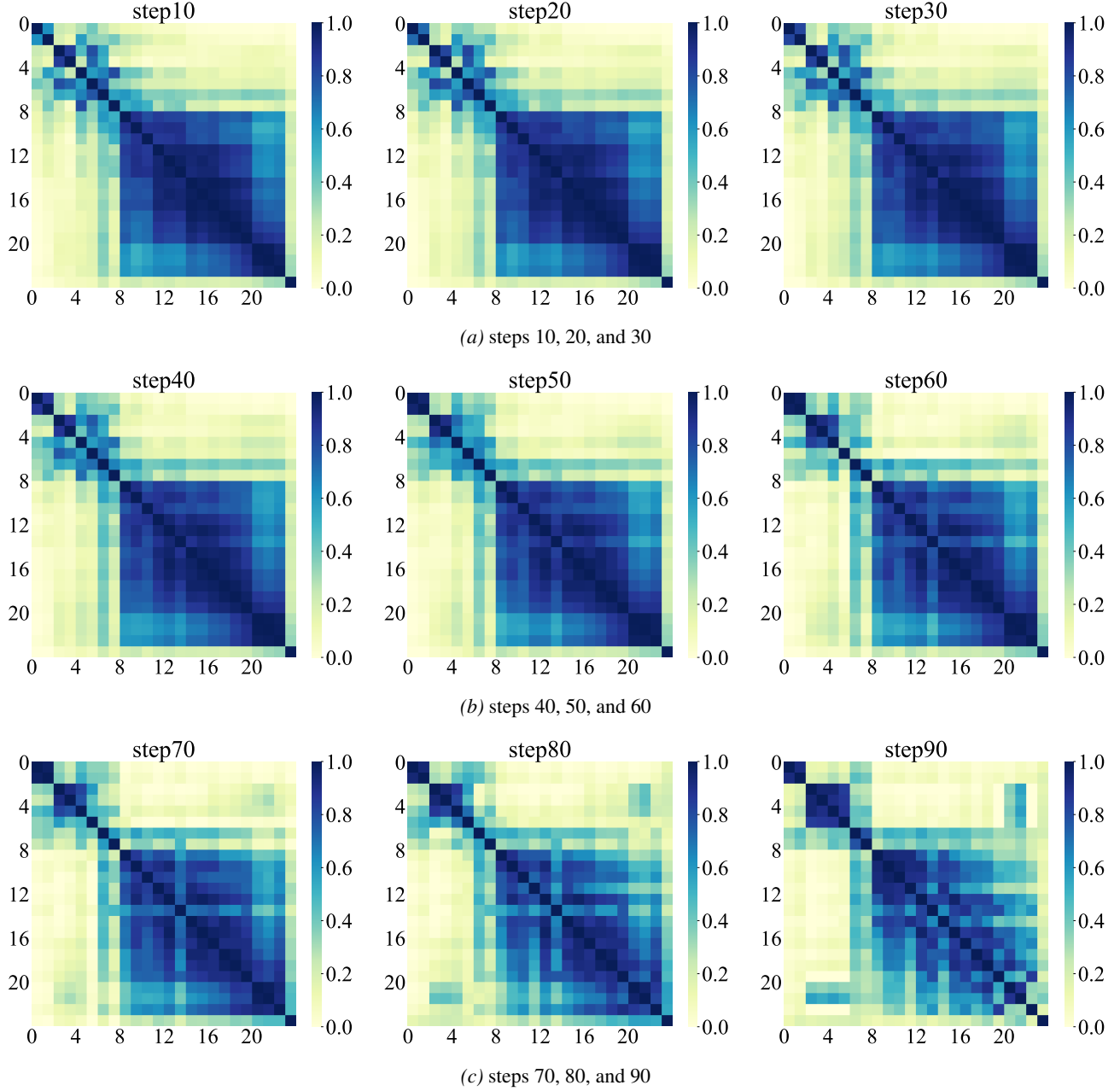


Figure 12. Redundancy across steps 10–90.

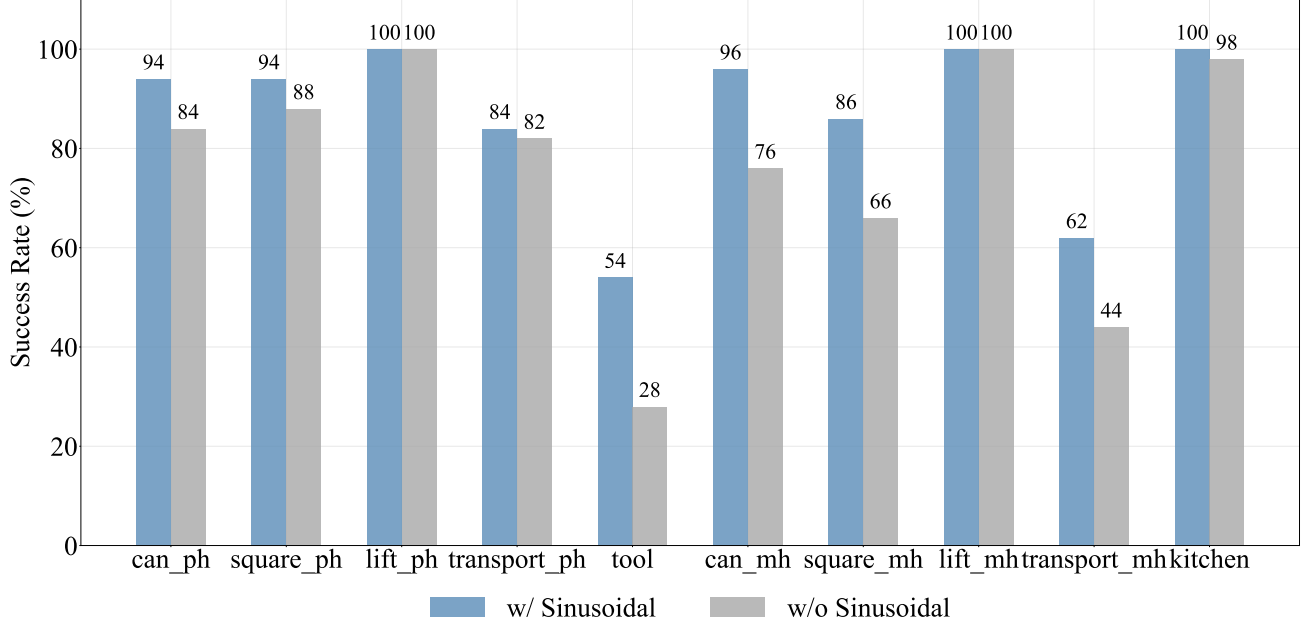


Figure 13. Task success rate (%) across multiple tasks for different positional encodings.

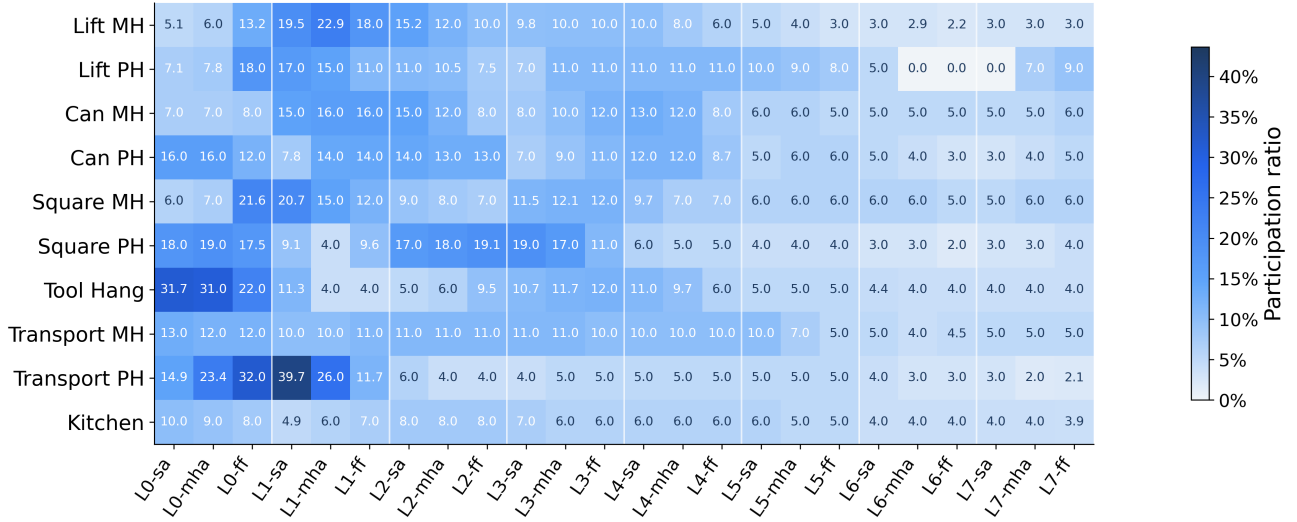


Figure 14. Per-block participation ratio across all 10 tasks, measured from closed-loop environment rollouts. Rows correspond to tasks and columns correspond to transformer sub-blocks within decoder layers.

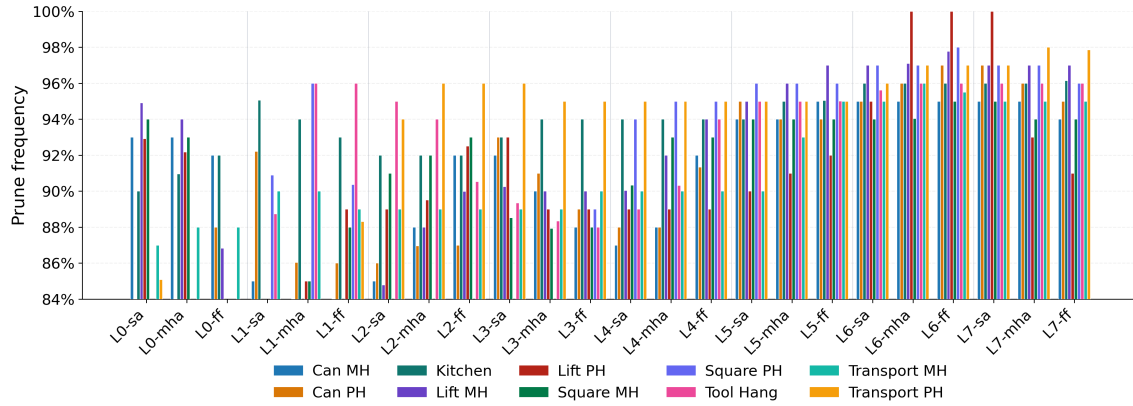


Figure 15. Block-level prune frequency across all 10 tasks. Each bar shows the fraction of rollout steps in which a given block is pruned.